

A Proof Governance Kernel with Machine-Checked Safety Invariants

Croft Adams¹

Independent Research
theadamsfamily1981@gmail.com

Abstract. Long-horizon mathematical proof campaigns routinely involve claims at different evidence levels: some unconditionally proved, others conditional on open conjectures, others supported only by heuristic computation. Misclassifying conditional or heuristic claims as unconditional is a common workflow failure mode in both human and AI-assisted mathematics.

We present a *proof governance kernel*: a small, typed state machine that tracks domain lifecycle (active, dropped, locked, reactivated), certificate strength (proof, reduction, computation, conditional, heuristic), and lock permanence (proven, conditional, heuristic). We formalize the kernel in Lean 4 and machine-check five safety invariants: (1) proven locks are permanent, (2) soft certificates cannot produce proven locks, (3) hard certificates are strictly stronger than soft ones, (4) every proven lock requires prior evidence, and (5) no lifecycle phase can be skipped.

We demonstrate the kernel on two case studies: tracking conditional versus unconditional status in a number-theoretic proof campaign (the generalized Fermat equation), and preventing circular dependencies in proof DAGs. Unlike build-level or session-level governance, our kernel operates at claim-level granularity, governing epistemic status across heterogeneous evidence sources. All Lean 4 sources compile without errors and are provided as a reproducibility artifact.

Keywords: proof engineering · Lean 4 · formal verification · governance · certificate systems · soundness

1 Introduction

Mathematical research, whether conducted by humans or AI systems, proceeds through claims of varying confidence. A conjecture supported by numerical evidence differs fundamentally from a peer-reviewed theorem, yet both may coexist in a single research programme. When proofs span months or years and involve multiple contributors—increasingly including machine learning systems whose outputs may be plausible but unsound [10]—the risk of *unsound claim promotion* becomes acute: a conditional result is inadvertently treated as unconditional, or a heuristic observation is locked as settled.

We propose a lightweight *proof governance kernel* that addresses this risk through three mechanisms:

1. **Typed certificates.** Every state transition carries a certificate whose type (proof, reduction, computation, conditional, or heuristic) is machine-checked. Certificate types are ordered by strength and partitioned into *hard* (proof, reduction, computation) and *soft* (conditional, heuristic).
2. **Typed locks.** When a domain is resolved, it receives a lock whose type (proven, conditional, heuristic) determines whether it can be reopened. The lock type must be compatible with the certificate type: only hard certificates can produce proven locks.
3. **Lifecycle enforcement.** A domain must pass through mandatory intermediate states (active $\rightarrow$ dropped $\rightarrow$ locked), preventing shortcuts that bypass evidence requirements.

We formalize this kernel in Lean 4 (v4.14.0) and prove five safety theorems entirely within Lean’s kernel, requiring no axioms beyond the Lean 4 type theory. The formalization comprises approximately 350 lines of Lean across three files.

Contributions.

- A formal model of proof state governance with typed certificates and locks (§2).
- Five machine-checked safety invariants (§3).
- Two case studies demonstrating practical application (§4, §5).
- A reproducibility artifact with build instructions (§7).

2 The Kernel Model

2.1 Certificate Types

We define five certificate types, ordered by decreasing strength:

Definition 1 (Certificate Type). *A certificate type is one of five values, ordered by decreasing strength: **proof** (5), **reduction** (4), **computation** (3), **conditional** (2), **heuristic** (1). A certificate type is hard if it is **proof**, **reduction**, or **computation**; otherwise it is soft.*

In Lean 4:

```

1 inductive CertificateType where
2   | proof | reduction | computation | conditional | heuristic
3
4 def CertificateType.strength : CertificateType → Nat
5   | .proof => 5 | .reduction => 4 | .computation => 3
6   | .conditional => 2 | .heuristic => 1
7
8 def CertificateType.isHard : CertificateType → Bool
9   | .proof => true | .reduction => true | .computation => true
10  | .conditional => false | .heuristic => false

```

2.2 Lock Types

Definition 2 (Lock Type). A lock type is one of:

- *proven*: permanent (cannot reopen),
- *conditional*: reopenable upon condition resolution,
- *heuristic*: reopenable with new tools.

```

1 def LockType.canReopen : LockType → Bool
2   | .proven => false
3   | .conditional => true
4   | .heuristic => true

```

2.3 Domain States and Transitions

Definition 3 (Domain State). A domain occupies one of four states: *active* (under investigation), *dropped*(*ct*) (reduced with certificate type *ct*), *locked*(*lt*) (resolved with lock type *lt*), or *reactivated* (reopened from a non-proven lock).

Definition 4 (Valid Transition). The permitted transitions are defined inductively:

$$\text{active} \rightarrow \text{dropped}(ct) \quad \text{for any } ct \quad (1)$$

$$\text{dropped}(ct) \rightarrow \text{locked}(lt) \quad \text{for any } lt \quad (2)$$

$$\text{locked}(lt) \rightarrow \text{reactivated} \quad \text{iff } lt.\text{canReopen} = \text{true} \quad (3)$$

$$\text{reactivated} \rightarrow \text{active} \quad (4)$$

2.4 Lock Permission

Definition 5 (Permitted Lock). A certificate type *ct* permits a lock type *lt* according to:

- *permittedLock*(*ct*, *proven*) iff *ct.isHard* = *true*;
- *permittedLock*(*ct*, *conditional*) iff *ct* = *conditional* or *ct.isHard* = *true*;
- *permittedLock*(*ct*, *heuristic*) always.

3 Safety Theorems

All governance kernel theorems below are machine-checked in Lean 4 with no `sorry` and no additional axioms. (The case study in §4 includes one intentional `sorry` marking conditional analytic content; it is not used by any kernel theorem.)

Theorem 1 (Proven Lock Permanence). No valid transition exits the proven-locked state:

$$\forall s. \neg \text{ValidTransition}(\text{locked}(\text{proven}), s).$$

Proof. By case analysis on `ValidTransition`. The only constructor producing a transition from `locked(lt)` is `lockToReactivate`, which requires `lt.canReopen = true`. Since `proven.canReopen = false`, no constructor applies.

Theorem 2 (No Premature Proven Lock). *Soft certificates cannot produce proven locks:*

$$\begin{aligned} &\neg \text{permittedLock}(\text{heuristic}, \text{proven}) \\ &\neg \text{permittedLock}(\text{conditional}, \text{proven}) \end{aligned}$$

Proof. By unfolding `permittedLock` and `isHard`: both `heuristic.isHard` and `conditional.isHard` reduce to `false`, contradicting the requirement for a proven lock.

Theorem 3 (Strength Gap). *Every hard certificate type has strictly greater strength than every soft certificate type:*

$$\begin{aligned} \forall ct_h ct_s. ct_h.isHard = true \wedge ct_s.isHard = false \\ \implies ct_h.strength > ct_s.strength. \end{aligned}$$

Proof. Hard types have strength ≥ 3 and soft types have strength ≤ 2 , established by exhaustive case analysis on the five constructors.

Theorem 4 (Evidence Requirement). *Any transition chain from `active` to `locked(proven)` must include an intermediate `dropped` state:*

$$\begin{aligned} \text{CertChain}(\text{active}, \text{locked}(\text{proven})) \implies \\ \exists ct. \text{Nonempty}(\text{CertChain}(\text{active}, \text{dropped}(ct))). \end{aligned}$$

Proof. By case analysis on the chain. A single-step chain from `active` to `locked(proven)` is impossible (no matching constructor). A multi-step chain must begin with `activeToDrop(ct)`, giving the witness.

Theorem 5 (No State Skipping). *The `active` state cannot transition directly to any locked state:*

$$\forall lt. \neg \text{ValidTransition}(\text{active}, \text{locked}(lt)).$$

Proof. By case analysis: no constructor of `ValidTransition` has source `active` and target `locked(lt)`.

4 Case Study I: Conditional Certificates in Number Theory

We instantiate the governance kernel on the generalized Fermat equation $a^p + b^q = c^r$, specifically tracking the distinction between conditional and unconditional results.

Background. Define $\lambda = 1/p + 1/q + 1/r$. When $\lambda < 1$, the equation has at most finitely many coprime solutions by the Darmon–Granville theorem [3], which invokes Faltings’ theorem [4]. A standard application of the ABC conjecture [7,8] yields a shorter finiteness argument, but this is *conditional* on the unproved ABC conjecture.

Certificate Assignment. In our kernel, the Darmon–Granville result receives a **proof** certificate (unconditional, peer-reviewed). The ABC-based argument receives a **conditional** certificate with an explicit blocking seam: “ABC conjecture unproved.”

Governance Enforcement. We verify in Lean 4:

- The ABC certificate is not hard: `abcBealCertificate.isHard = false`.
- It cannot support a proven lock:
 $\neg \text{permittedLock}(\text{conditional}, \text{proven})$.
- The Darmon–Granville certificate *can* support a proven lock:
`permittedLock(proof, proven)`.

Signature Verification. We additionally verify regime classifications for specific signatures using Lean’s `native_decide` tactic:

- (2, 2, 2): spherical ($\lambda = 3/2 > 1$).
- (3, 3, 3), (2, 3, 6), (2, 4, 4): euclidean ($\lambda = 1$).
- (3, 5, 7): hyperbolic ($\lambda = 71/105 < 1$).

5 Case Study II: Circular Dependency Prevention

A second failure mode in proof campaigns is *circular dependency*: claim *A* depends on claim *B*, which depends on *A*. Our kernel prevents this structurally.

Structural Argument. Define a rank function: `active` $\mapsto 0$, `dropped`($_$) $\mapsto 1$, `locked`($_$) $\mapsto 2$, `reactivated` $\mapsto 0$. By Theorem 5, forward transitions (excluding reactivation) strictly increase rank. Since the rank space is finite, any forward-only chain terminates. Reactivation resets rank to zero but requires the lock to be non-proven (Theorem 1).

Mechanized Impossibility. We verify in Lean 4 that the proven-locked state is a dead end:

```

1 -- No transition leaves proven-locked state
2 theorem proven_is_terminal :
3    $\forall$  (s : DomainState),
4      $\neg$  ValidTransition (.locked .proven) s := by
5   intro s h; cases h with
6   | lockToReactivate lt hcan =>
7     simp [LockType.canReopen] at hcan

```

```

8
9 -- Cannot skip active directly to locked
10 theorem no_skip_active_to_locked :
11   ∀ (lt : LockType),
12   ¬ ValidTransition .active (.locked lt) := by
13   intro lt h; cases h

```

Corollary 1 (No Proven Lock Cycle). *A domain that reaches `locked(proven)` cannot return to `active` via any sequence of valid transitions.*

Proof. Immediate from `proven_is_terminal`: no constructor of `ValidTransition` matches the source `locked(proven)`.

Conditional and Heuristic Cycles. Conditional and heuristic locks *do* permit cycles (`active` $\rightarrow$ `dropped` $\rightarrow$ `locked` $\rightarrow$ `reactivated` $\rightarrow$ `active`), which is intentional: new tools or resolved conditions should allow revisiting a domain. The kernel guarantees that such cycles never produce proven locks from soft evidence (Theorem 2).

6 Related Work

Proof Engineering in ITP. The challenge of maintaining large formal developments is well-documented. Gross et al. [5] address proof-engine scaling in Coq through performant rewriting primitives. Sarracino et al. [2] survey Coq users and identify proof brittleness and workflow safety as persistent community concerns. Isabelle’s session model [6] provides build-level governance through document preparation and dependency tracking. Our work operates at a different granularity: we govern the *epistemic status* of claims, not the build process, though both concerns are complementary.

Certificate-Based Approaches. Proof certificates appear in several contexts: Coq’s `Qed` command verifies a proof term against a type; SAT/SMT solvers produce checkable resolution traces; and proof-producing compilers (e.g., CakeML) carry correctness witnesses through compilation. Our certificates are coarser-grained: they classify evidence *type* rather than carrying the evidence itself. This tradeoff sacrifices completeness for lightweight governance that can span heterogeneous proof methods (formal, computational, conditional, heuristic).

AI-Assisted Mathematics. Machine learning systems for theorem proving—including tactic prediction [1] and autoformalization—are increasingly powerful but can produce plausible yet unsound output, a phenomenon studied under “hallucination” [10]. Our kernel is source-agnostic: it enforces that AI-generated heuristic evidence cannot be silently promoted to proven status, regardless of how convincing it appears.

7 Artifact and Reproducibility

The Lean 4 formalization is available at [repository URL]¹. The artifact comprises:

- `Governance.lean`: definitions and core lock/permission theorems (82 lines).
- `Lambda.lean`: signature classification and case study I (148 lines).
- `Soundness.lean`: safety theorems (98 lines).

Build. Install Lean 4 v4.14.0 via `elan`, then run `lake build`. The toolchain is pinned via `lean-toolchain` (content: `leanprover/lean4:v4.14.0`). The build is deterministic given this toolchain and completes with zero errors.

Sorry isolation. The single `sorry` in `Lambda.lean` marks the analytic content of the ABC argument, which is conditional by design. It is not imported or used by `Soundness.lean`; all five kernel safety theorems (Theorems 1–5) are `sorry-free`.

8 Conclusion and Future Work

We have presented a proof governance kernel that enforces typed certificates and locks on mathematical proof campaigns, preventing unsound claim promotion. Five safety invariants are machine-checked in Lean 4 with no additional axioms.

Future directions include:

- **Fine-grained conditionality.** Extending the certificate hierarchy to distinguish GRH-conditional, ABC-conditional, and computational results, with composition rules that track maximum conditionality through chains.
- **Integration with proof assistants.** Embedding the kernel as a Lean 4 meta-programming plugin that automatically classifies tactic-generated proofs.
- **Empirical evaluation.** Applying the kernel to a real proof campaign (e.g., the Liquid Tensor Experiment or Fermat’s Last Theorem formalization) and measuring whether it catches misclassified claims.

References

1. L. Blaauwbroek, J. Urban, H. Geuvers. Tactic learning and proving for the Coq proof assistant. In: *LPAR-23*, EPIc Series, 2020.
2. J. Sarracino et al. Lessons for interactive theorem proving researchers from a survey of Coq users. *J. Autom. Reason.*, 2025.
3. H. Darmon, A. Granville. On the equations $z^m = F(x, y)$ and $Ax^p + By^q = Cz^r$. *Bull. London Math. Soc.*, 27(6):513–543, 1995.
4. G. Faltings. Endlichkeitssätze für abelsche Varietäten über Zahlkörpern. *Invent. Math.*, 73(3):349–366, 1983.
5. J. Gross, A. Chlipala, et al. Towards a scalable proof engine: A performant prototype rewriting primitive for Coq. *J. Autom. Reason.*, 2024.

¹ To be provided upon publication.

6. Isabelle Team. Isabelle system manual: Theory sessions. <https://proofcraft.systems/isabelle/dist/library/Doc/System/Sessions.html>, 2025.
7. D. W. Masser. Open problems. In: *Proc. Sympos. Analytic Number Theory*, London, 1985.
8. J. Oesterlé. Nouvelles approches du “théorème” de Fermat. *Sém. Bourbaki*, exp. 694, 1988.
9. D. Ratcliffe, B. Grechuk. Generalised Fermat equation: A survey of solved cases. *Exposition. Math.*, 43(4), 2025.
10. S. Farquhar, J. Kossen, L. Kuhn, Y. Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.
11. S. Dahmen. Generalized Fermat equations with three distinct prime exponents. MPIM Number Theory Lunch Seminar, Apr. 2, 2025.